ARAŞTIRMA SORULARI

1.Kullanıcı şifresi neden düz metin olarak saklanmamalıdır?

-Anında Veri İfşası:
–Sistem veya veritabanı ele geçirildiğinde(veri ihlali),saldırganlar şifreleri çözmek için zaman kaybetmez;tüm kullanıcıların parolalarını doğrudan okunabilir halde görür.

-Zincirleme Hesap Güvensizliği:
–Kullanıcılar genellikle aynı e-posta ve şifre kombinasyonunu farklı platformlarda(e-posta,bankacılık,sosyal medya)kullandığı için,tek bir sitedeki sızıntı diğer tüm hesaplarının da ele geçirilmesine neden olur.

-Gizlilik ve Güven İhlali:
–Şirket çalışanlarının veya kötü niyetli kişilerin veritabanına erişerek kullanıcıların şifrelerini görmesi engellenmez.

-Yasal ve İtibarı Zararlar:
–Şifre güvenliğine dair standartların(KVKK,GDPR,OWASP vb.)ihlal edilmesi şirketlere ağır yasal cezalar ve büyük bir itibar kaybı yaşatır.

2.Hash nedir?Encryption ile yanı şey midir?

Aynı şey değildir; her ikisi de veri güvenliği için kullanılsa da temel amaçları ve çalışma şekilleri tamamen farklıdır.

Hash (karma fonksiyonu)Nedir?
-Tanım:
–Herhangi uzunluktaki bir veriyi alan ve bunu sabit uzunlukta,benzersiz bir karakter dizisine(özet değere)dönüştürülen tek yönlü bir matematiksel işlemdir.

-Kullanım amacı:
–Veri bütünlüğünü doğrulamak ve parolaları güvenli bir şekilde saklamak için kullanılır.Örneğin,bir dosyanın indirilirken bozulup bozulmadığını kontrol etmek veya veritabanında şifrelerin kendisi yerine hash özetlerini tutmak bu amaçtır.

Encryption(şifreleme)Nedir?
-Tanım:
–Okunabilir bir veriyi(plain text),bir anahtar kullanarak kimsenin anlayamayacağı karmaşık bir formata(cipher text)dönüştürülen çift yönlü bir işlemdir.

-Kullanım anlayamayacağı:
–Verinin gizliliğini korumak ve güvenli şekilde iletilmesini veya saklanmasını sağlamaktadır.Örneğin,uçtan uca mesajlaşma uygulamaları veya HTTPS bağlantıları veriyi şifreler.

3.Input validation neden yapılır?

-Güvenlik Açıklıklarını Önlemek:
–SQL ınjection,XSS(Cross-site scripting)gibi zararlı kod enjeksiyonu saldırılarının sisteme sızmasını engeller.

-Veri Bütünlüğünü Sağlamak:
–E-posta adresine metin,telefon numarasına harf veya yaş alanına negatif sayı girilmesi gibi hatalı veri girişlerinin önüne geçer.

-Sistem Kararlılığını korumak:
–Beklenmeyen formtta veya çok uzun verilerin veritabanına veya arka uç(backend)kodlarına gidip uygulamayı çökertmesini(crash)engeller.

-Kullanıcı Deneyimini İyileştirmek:
–Kulanıcıya hatalı bir işlem yaptığında veya eksik bilgi girdiğinde anında net ve açıklayıcı uyarılar gösterir.

-Kaynak Tassarrufu Sağlamak:
–Geçersiz veya hatalı verilerin gereksiz yere veritabanı sorgularına veya maliyetli sunucu işlemlerine sokulmsını engeller.

4.SQL ınjection nedir?

Saldırganların bir web veri tabanı sorgularına kötü amaçlı SQL kodları ekleyerek veriyi ele geçirmesine,değiştirmesine veya silmesine olanak tanıyan yaygın bir siber güvenlik açığıdır.

5.XSS Nedir?

Cross-site scripting attacks(XSS) yani siteler arası komut dosyası çalıştırma saldırısı,bir bilgisayar korsanın,iyi huylu ve güvenilir olarak görülen bir web sayfasının içeriğine,genellikle istemci tarafı komut dosyası biçiminde kötü amaçlı kod enjekte etmesiyle oluşur.Kötü amaçlı komut dosyası genellikle,JavaScript ve HTML olan istemci tarafı programlama dillerinde yazılır.

6.Apı key,parola veya bağlantı bilgileri neden repository'ye yazılmamlıdır?

1.Yetkisiz Erişim ve Veri İhlalleri
-Genel Değişiklik:
–Deponuz(örneğin GitHub,GitLub veya Bitbucket'ta)herkese açık(public)ise,kodunuzu gören herkes bu şifreleri alabilir.

-Otomatik Botlar:
–Siber saldırganlar,GitHub gibi platformları saniyeler içinde tarayan otomatik botlar kullanır.Kodunuzu paylaştığınız an botlar bu anahtarları yakalar ve veritabanınızı silebilir,verilerinizi çalabilir veya hesabınız üzerinden binlerce dolarlık bulut faturası çıkarabilir.

2.Gizli Depoların(Private repository)Güvenilmezliği
-Erişim Esnekliği:
–Kod deposunun şu an gizli olması,sonsuza kadar güvende olduğu anlamına gelmez.Depoya erişimi olan bir iş arkadaşınızın hesabı çalınabilir veya şirket gelecekte bu projeyi açık kaynaklı hale getirmeye karar verebilir.

-Geçmişi Silmek İmkansızdır:
–Git gibi versiyon kontrol sistemleri,yaptığınız her değişikliği kaybeder.Koddaki şifreyi silip yeni bir commit(gönderim)yapsanız bile,şifre kodun geçmişinde(history)kalmaya devam eder.Geçmişten tamamen temizlemek ise oldukça zordur.

-Esneklik ve Yönetim Zorluğu
-Ortam Farklılıkları:
–Yerel bilgisayarınızda(Local),test sunucusunda(staging)ve canlı sistemde(production)kullanılan şifreler ve veritabanları farklı olmalıdır.Şifreleri koda gömmek,projenin farklı ortamlarda çalışmasını zorlaştırır.

-Şifre Değştirme Zorluğu:
–Bir parolanın sızdırılması durumunda veya periyodik olarak değiştirilmesi gerektiğinde,tüm kodu yeniden düzenleyip,commit atıp,sunucuya tekrar yüklemek(deploy)gerekir.

7.Secret kavramı neyi iafe eder?

Yetkisiz kişilerin erişmemesi gereken,hassas ve gizli kimlik doğrulama bilgilerini ifade eder.

GÜN SONUNDA TESLİM EDECEĞİNİZ NOT:

1.Bir kullanıcı kayıt sistemi tasarlarken dikkat edeceğiniz güvenlik maddelerini listeleyin:

1.Güçlü Şifre kullanımı:
–Kullanıcıların tahmin edilmesi zor şifreler oluşturulması sağlanmalıdır.

2.Şifreleri Koruma:
–Şifreler veri tabanında açık metin olarak tutulmamalı,güvenli yöntemlerle hashlenmelidir.

3.Kimlik Doğrulama:
–Kullanıcının kimliği giriş sırasında kontrol edilmelidir.

4.Yetkilendirme:
–Her kullanıcı yalnızca izin verilen işlemleri yapabilmelidir.

5.Veri Doğrulama:
–Kullanıcıdan alınan bilgiler kontrol edilmeli ve hatalı girişler engellenmelidir.

6.Veri Tabanı Güvenliği:
–SQL Injection gibi saldırılara karşı parametreli sorgular kullanılmalıdır.

7.HTTPS Kullanımı:
–Kullanıcı bilgileri internet üzerinden şifreli olarak gönderilmelidir.

8.Giriş denemelerini sınırlama:
–Çok sayıda başarısız giriş denemesi kontrol altına alınmalıdır.

9.Kişisel Verileri Koruma:
–Kullanıcının kişisel bilgileri izinsiz kişilerle paylaşılmamalıdır.

10.Güvenli Oturum Yönetimi:
–Otuurm bilgileri korunmalı ve çıkış yapıldığında oturum sonlandırılmalıdır.

11.E-posta Doğrulama:
–Gerekli durumlarda kayıt sırasında e-posta adresinin doğruluğu kontrol edilmelidir.

12.Hata Mesajlarını Güvenli Gösterme:
–Hata mesajlarında şifre veya sistemin hassas bilgileri açığa çıkarılmamalıdır.

2.Repository'ye kesinlikle koymamanız gereken bilgilere örnekler bulun.

1.Şifre:
–"Okul12345!"

2.API Anahtarı:
–"API_KEY=abc123gizli"

3.Veritabanı Şifresi:
–"DB_PASSWORD=MySecret123"

4.T.C. Kimlik Numarası:
–"Bir kişiye ait gerçek kimlik numarası."

5.Özel Anahtar:
–"SSH ile erişim sağlayan gizli anahtar dosyası."

6.Erişim Tokenı:
–"GitHub hesabına erişim sağlayan kişisel token."

7.Gizli Yapılandırma Dosyası:
–"Şifreler içeren .env dosyası."

8.Kişisel Bilgiler:
–"Müşterilerin adresleri,telefon numaraları ve özel bilgileri."

3.SQL Injection ve XSS'in hangi katmanda ortaya çıktığını kendi şemanız üzerinde göstermeye çalışın.

SQL Injection ve XSS

Hangi katmanda ortaya çıkar?



Kullanıcı / Tarayıcı

Kulanıcı,formda bilgi girer ve isteği gönderir.

Bu aşamada kullanıcı güvenilmeyen veri girebilir.(ör.zararlı kod, SQL sorgusu vb.)



Sunucu / Uygulama Katmanı

Gelen istek işlenir,doğrulama yapılır ve gerekli işlemler gerçekleştirilir.



SQL Injection

veritabanı katmanı

Uygulamanın veritabanına gönderdiği sorguların güvenli hazırlanmasıyla ortaya çıkar.



XSS

Tarayıcı Katmanı

Güvenilmeyen içerik,güvenli biçimde işlenmeden sayfada gösterildiğinde ortaya çıkar.